

从游戏迷茫到精准洞察： 构建智能游戏推荐社区

基于Django的智能游戏推荐系统设计与实现

汇报人：马腾

指导教师：孟庆轩

专业：软件工程（阿里云校企合作班）



市场繁荣下的“选择悖论”

游戏市场空前繁荣，但玩家发现好游戏的难度不降反升。

市场规模



超过3000亿

2024年中国游戏市场收入

内容爆炸



超过2万款

新游戏上架数量

玩家痛点

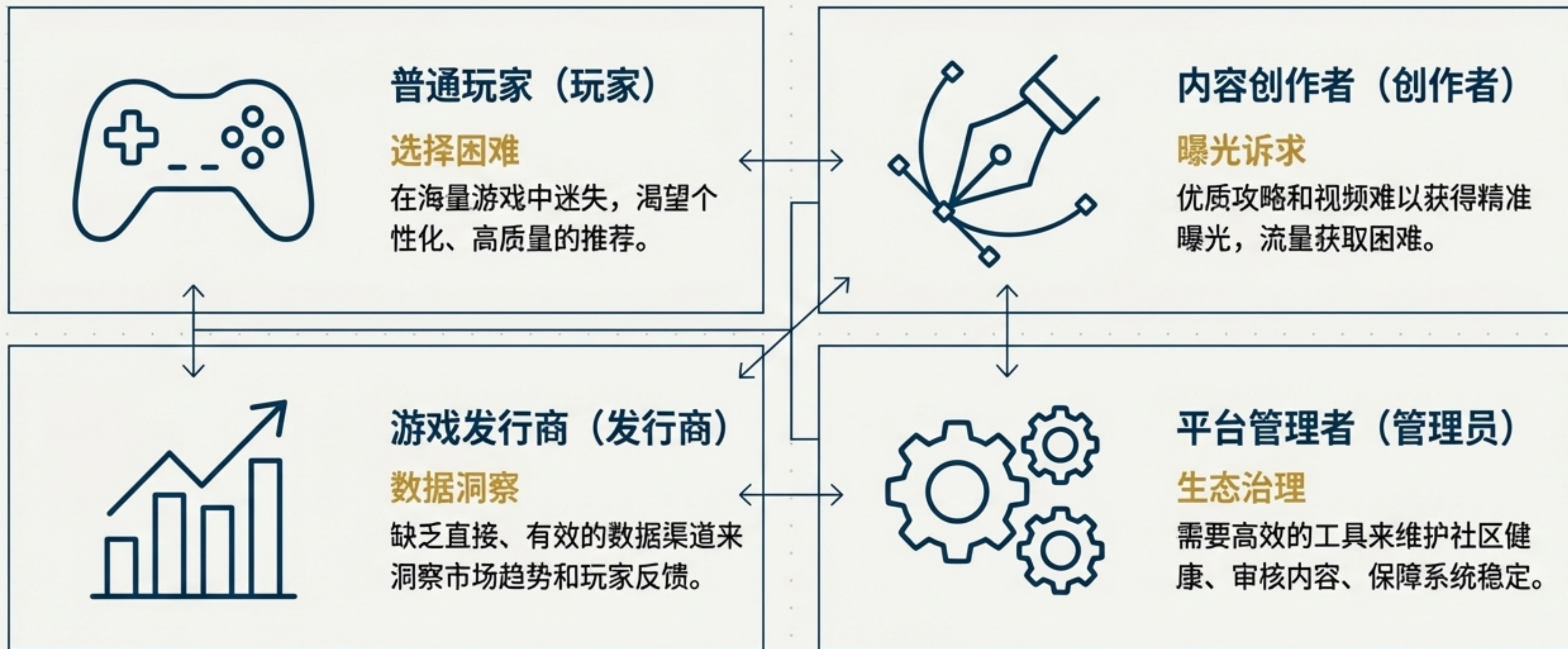
选择困难



现有社区推荐功能单一，多为简单的热度排序，无法满足个性化需求。

一个挑战，多方困局：构建生态的必要性

游戏发现的难题影响着生态中的每一个参与者。



我们的愿景：连接玩家、内容与数据的智能生态平台

打造一个集精准推荐、数据分析、内容创作与社区互动于一体的多功能游戏社区平台。



1. 数据采集与深度分析

结合外部数据与社区动态行为，构建数据基础。



2. 核心推荐算法实现

设计并实现以热度权重为核心的推荐算法，解决“游戏荒”。



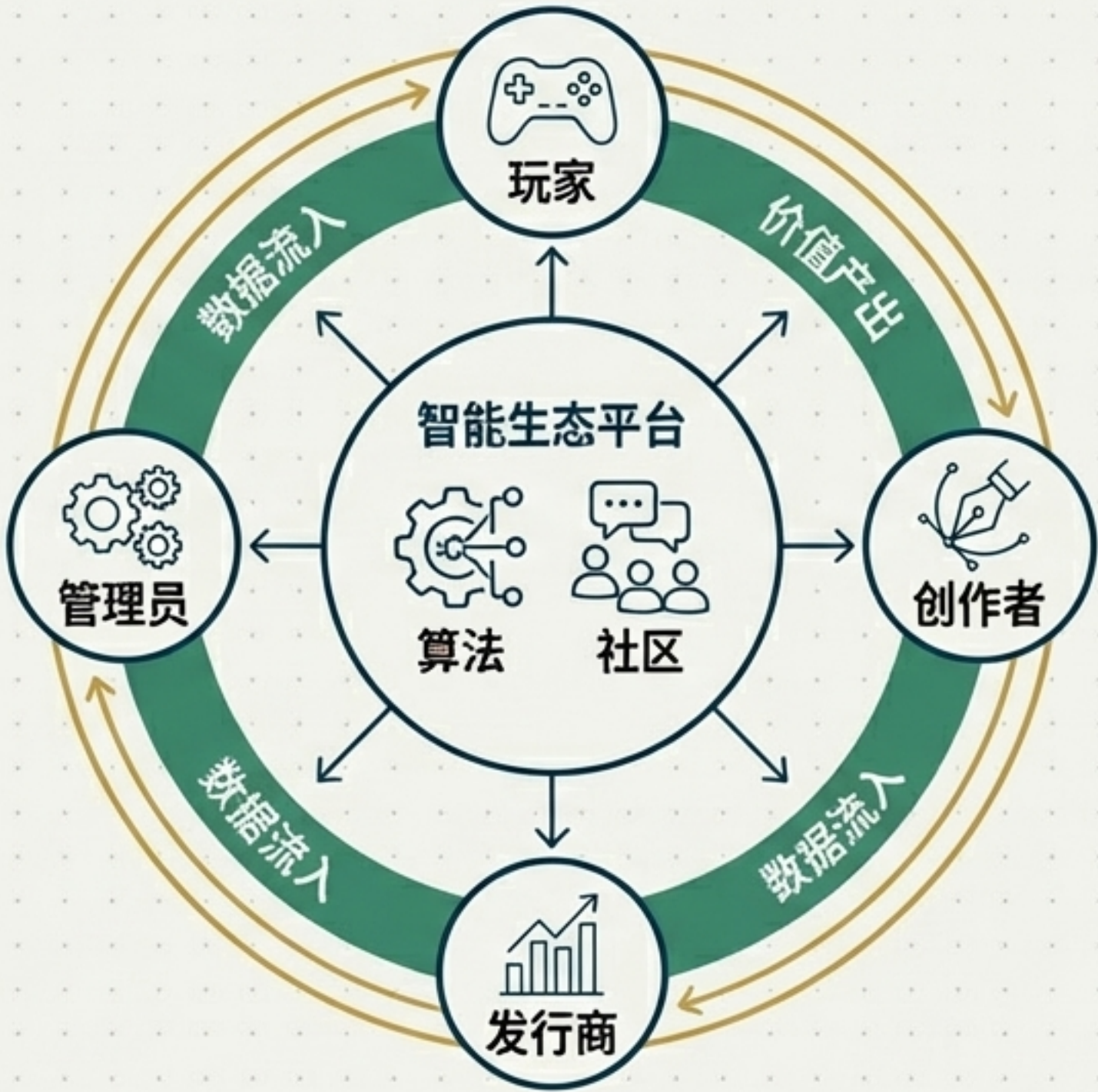
3. 多角色互动社区构建

建立连接玩家、创作者和发行商的良性互动生态。

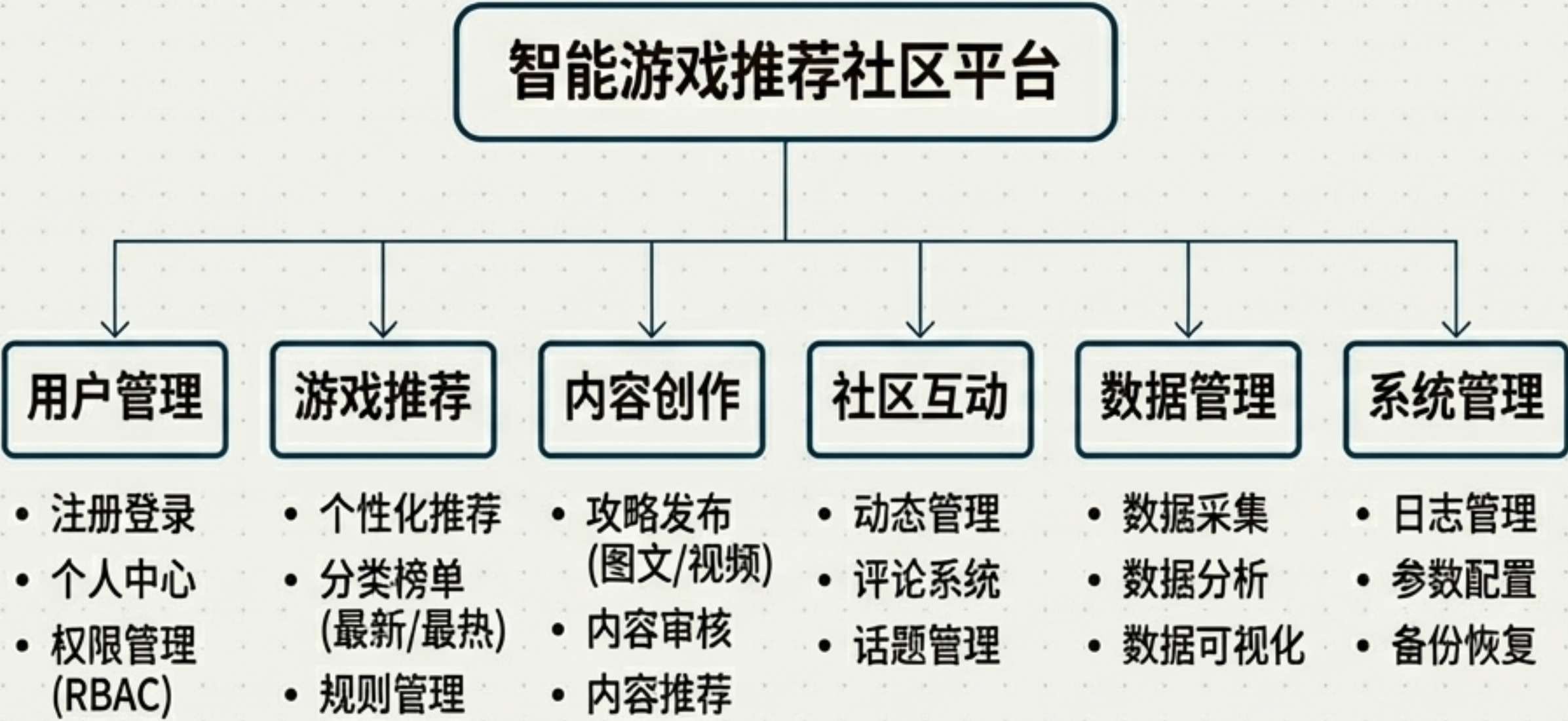


4. 可视化决策支持

将复杂数据转化为直观图表，为发行商提供决策支持。



系统功能全景：一体化平台的核心模块



服务于多方生态

-  玩家通过【游戏推荐】发现乐趣，在【社区互动】中分享体验。
-  创作者通过【内容创作】分享见解，获得曝光。
-  发行商借助【数据管理】的可视化分析洞察市场。
-  管理员使用【用户管理】和【系统管理】维护平台稳定。

系统的智能核心：融合长期价值与短期热度的推荐算法

兼顾游戏的长期市场考验（静态热度）与近期社区话题度（动态热度）。

1. 静态热度（ H_{static} ）： 衡量游戏长期积累的人气

$$H_{static} = w_d * D + w_f * F + w_r * R$$

- ↓ D: 下载数
- ❤ F: 关注数
- ☆ R: 评价数



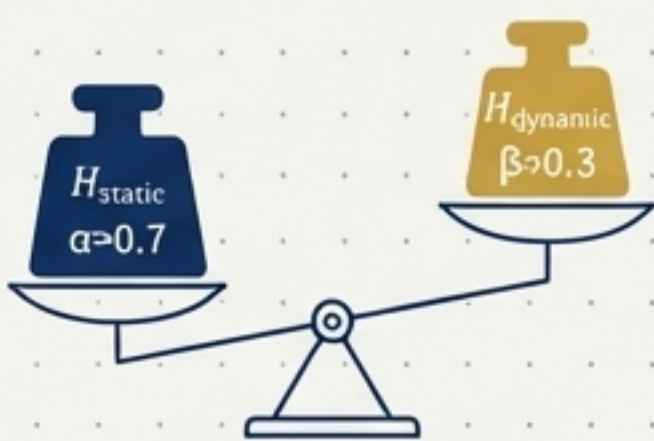
2. 动态热度（ $H_{dynamic}$ ）： 衡量游戏当前社区讨论热度

$$S_{post} = \frac{w_l * L + w_c * C}{T + 1}$$

- 引入时间衰减，单篇帖子分数累加
为总动态热度
- 👍 L: 点赞数
 - 💬 C: 讨论数
 - 🕒 T: 发布时长（小时）

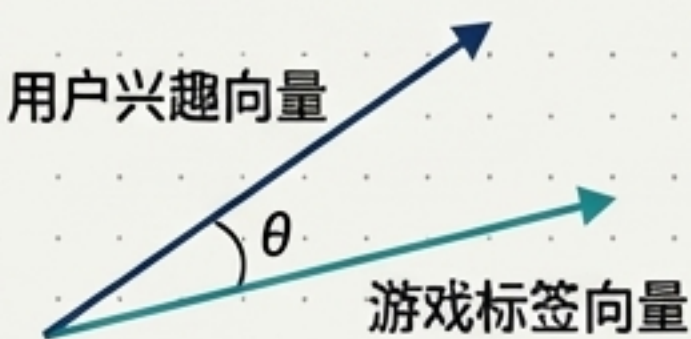
3. 总热度（ H_{total} ）：综合评价

$$H_{total} = \alpha * H_{static} + \beta * H_{dynamic}$$



个性化（余弦相似度）

通过计算“用户兴趣向量”与“游戏标签向量”的余弦相似度，实现“千人千面”的推荐。

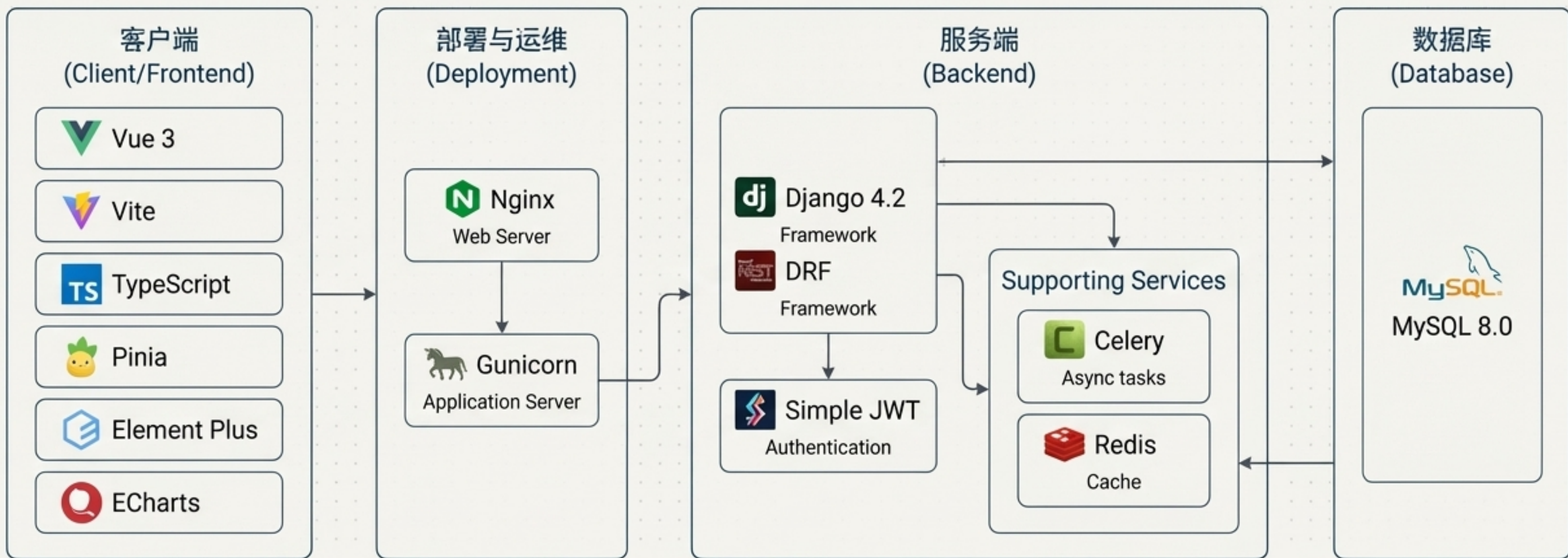


技术蓝图

铸造稳健、高效的智能生态系统

接下来，我们将深入探讨支撑整个平台运行的全栈技术架构、关键实现细节与数据流。

全栈技术架构：基于现代Web技术栈的分层设计



此架构实现了前后端分离、模块化和可扩展性。

前端实现：构建现代化、响应式的用户体验



统一请求管理

工具：“Axios”

实现：在 `src/utils/request.ts` 中封装实例，设置请求/响应拦截器。

亮点：自动注入 `Authorization: Bearer <token>`；统一处理401（令牌刷新/重定向）、400（字段错误提示）等HTTP状态码。



全局状态管理

工具：“Pinia”

实现：通过 `stores/user.ts` 管理用户信息、Token，并使用 `pinia-plugin-persistedstate` 实现持久化。

亮点：使用 `isLoggedIn`、`isAdmin` 等计算属性简化权限判断。



路由与权限控制

工具：“Vue Router”

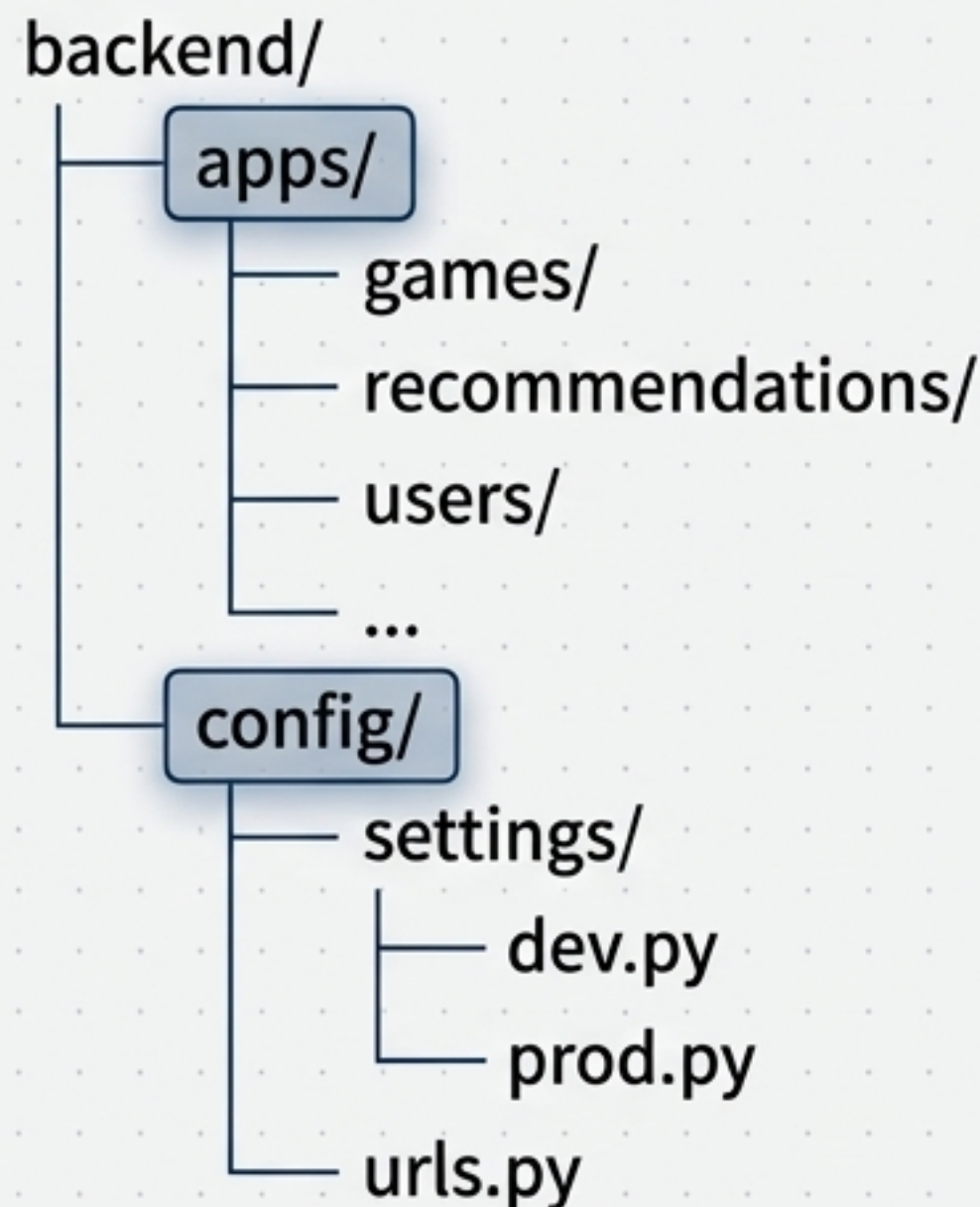
实现：在 `router/index.ts` 中配置路由表，并设置路由守卫，实现基于角色的页面访问控制。

`src/utils/request.ts` - 请求拦截器

```
1 import axios from 'axios';
2 import { useUserStore } from '@stores/user';
3
4 // Create Axios instance
5 const service = axios.create({
6   baseURL: import.meta.env.VITE_API_URL,
7   timeout: 5000,
8 });
9
10 // Request Interceptor
11 service.interceptors.request.use(
12   (config) => {
13     const userStore = useUserStore();
14     // Inject token if available
15     if (userStore.token) {
16       config.headers['Authorization'] = 'Bearer ' +
17     }
18     return config;
19   },
20   (error) => {
21     return Promise.reject(error);
22   }
23 );
```

后端实现：打造可扩展、高安全的业务核心

项目结构



模块化应用设计 (Apps)

结构：在 ``apps/`` 目录下组织业务子应用，如 ``games``、``recommendations``、``users`` 等。

优势：高内聚、低耦合，便于团队协作与功能迭代。



环境隔离与安全策略

实现：通过 ``config/settings/`` 目录下的 ``dev.py`` 和 ``prod.py`` 文件分离开发与生产环境配置。

亮点 (prod.py)：``DEBUG=False``，开启SSL重定向、安全Cookie、XSS过滤、HSTS等安全设置。



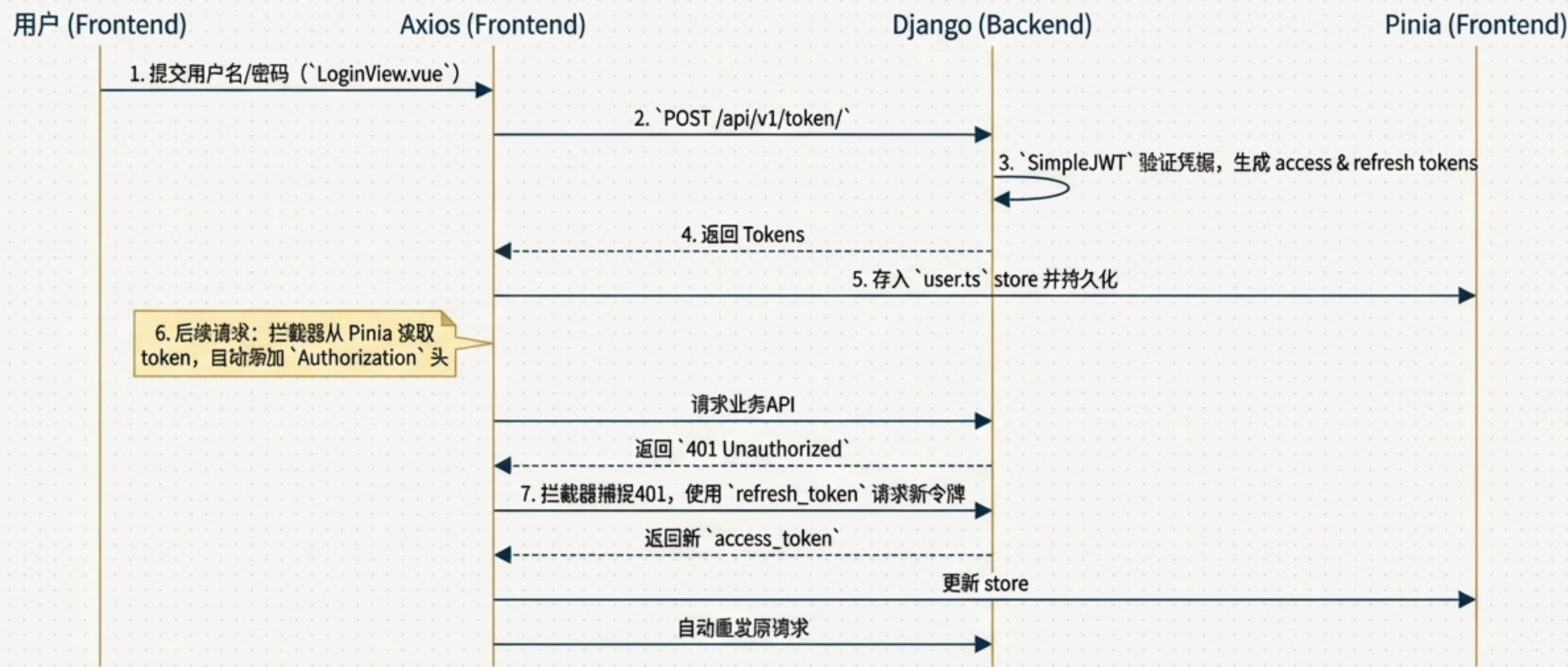
统一API设计与文档

框架：Django REST Framework (DRF)

实现：API统一前缀 ``/api/v1/``，使用 ``drf-spectacular`` 自动生成 Swagger 和 Redoc 接口文档。

数据流剖析（1）：JWT认证与无感刷新

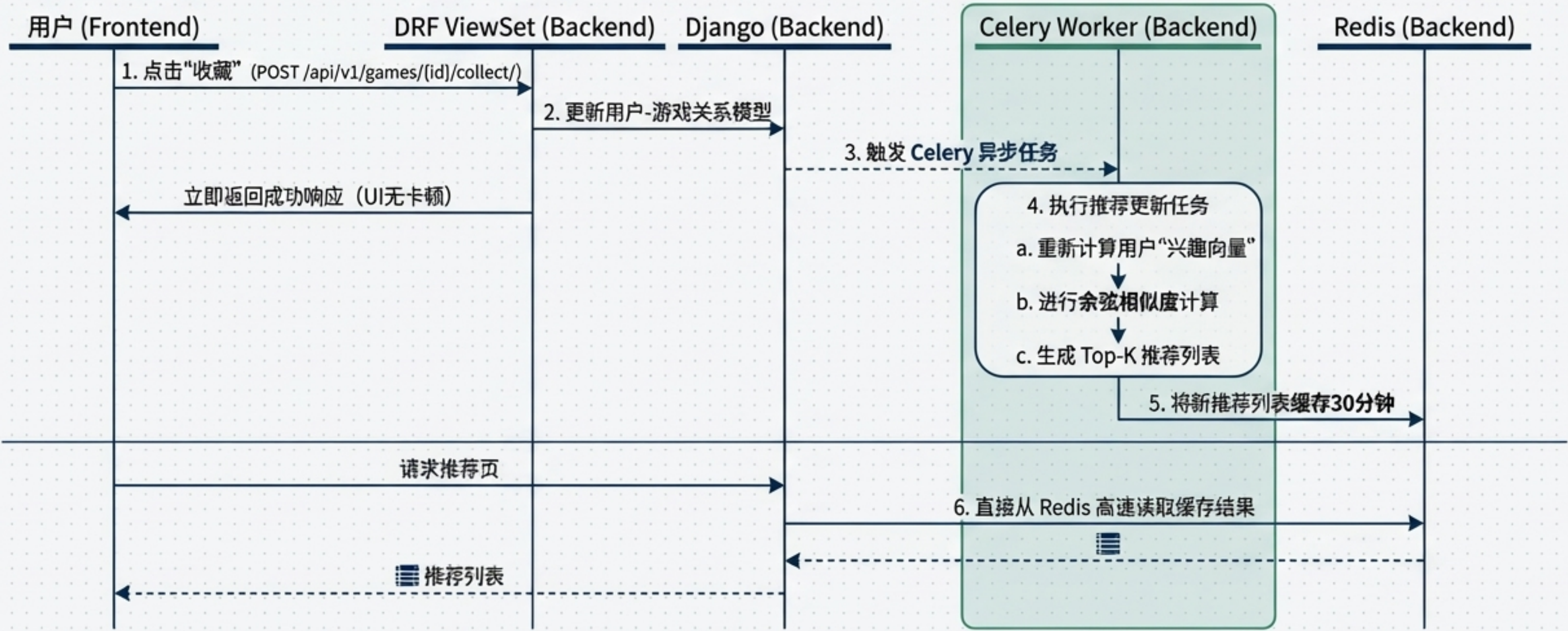
场景：用户登录流程



通过前后端协同，实现了安全、流畅的用户认证体验。

数据流剖析（2）：异步驱动的个性化推荐

场景：用户收藏游戏，触发推荐更新



采用Celery+Redis的异步架构，实现了用户操作与复杂计算的解耦，保证了前端的即时响应和后端的高效处理。

功能深潜：游戏截图管理的全栈实现

展示系统良好的可扩展性和完整的全栈开发流程。

后端实现

模型 (Model) : `GameScreenshot`

```
class GameScreenshot(models.Model):  
    """游戏截图模型"""  
    game = models.ForeignKey(  
        'Game',  
        on_delete=models.CASCADE,  
        related_name='screenshots',  
        verbose_name='所属游戏'  
    )  
    image = models.ImageField(  
        upload_to='game_screenshots/%Y/%m/',  
        verbose_name='截图文件'  
    )  
    title = models.CharField(  
        max_length=100,  
        blank=True,  
        verbose_name='截图标题'  
    )  
    order = models.PositiveIntegerField(  
        default=0,  
        verbose_name='排序权重'  
    )  
    # ... 其他字段和方法
```

视图 (ViewSet) : `GameScreenshotViewSet` 提供完整的 CRUD 操作, 并实现权限控制 (发行商/管理员可写, 所有人可读)。

路由 (URL) : 在 `GameViewSet` 中添加 `@action` 实现 `upload\_screenshot` 快捷上传接口。

前端实现

视图层 (`GameCreateEditView.vue`) : 集成 Element Plus 的 `el-upload` 组件, 支持批量上传、预览和删除。

API层: 在 `src/api/game.ts` 中新增 `uploadGameScreenshot`、`getGameScreenshots` 等方法。

项目总结与成果回顾

核心成就

解决了问题

成功设计并实现了一个智能游戏推荐社区，有效解决了玩家的“选择困难”，并为创作者和发行商提供了价值平台。

验证了技术

综合运用 Vue 3、Django、Celery 等现代Web技术，构建了一个稳健、可扩展的全栈应用。

实现了算法

实现了融合静态与动态热度的推荐算法，并通过异步任务和缓存机制保证了系统性能。

预期成果交付（源自开题报告1.4节）

✓ 完整的软件系统（前后端源码及数据库）

✓ 毕业设计报告（8000+字）

✓ 详细的部署文档

✓ 核心功能演示视频

结论：本项目达到了预期的设计目标，是一个功能完整、技术先进、具有实际应用价值的软件工程实践成果。

一个为所有参与者创造价值的生态系统

玩家视角

猜你喜欢



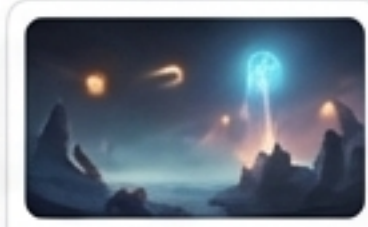
未来战记

动作 冒险 科幻



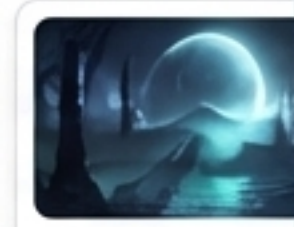
星际探索

动作 科幻



幻想之旅

动作 冒险 科幻



幻想之旅

猜你喜欢



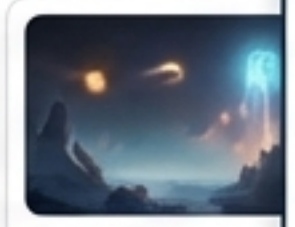
未来战记

动作 冒险



星际探索

动作 冒险 科幻



幻想之旅

动作 冒险

内容创作者视角

曝光量



12.8k

点赞数



972

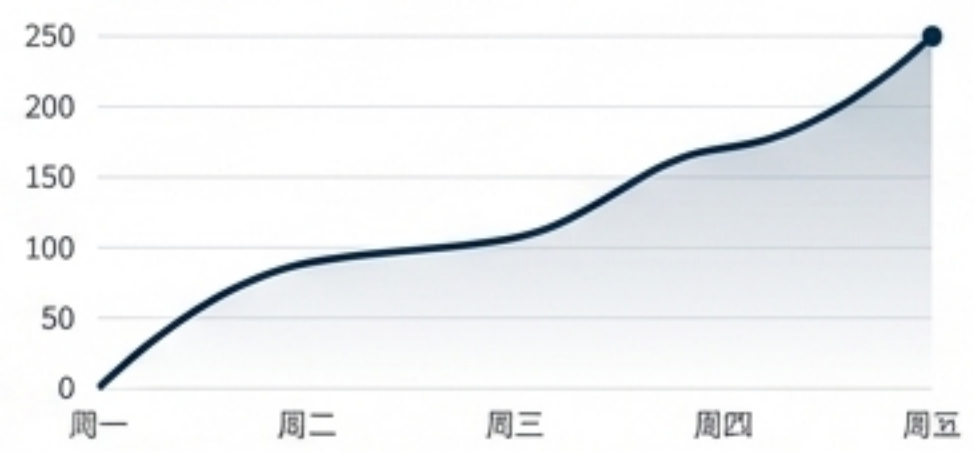
评论数



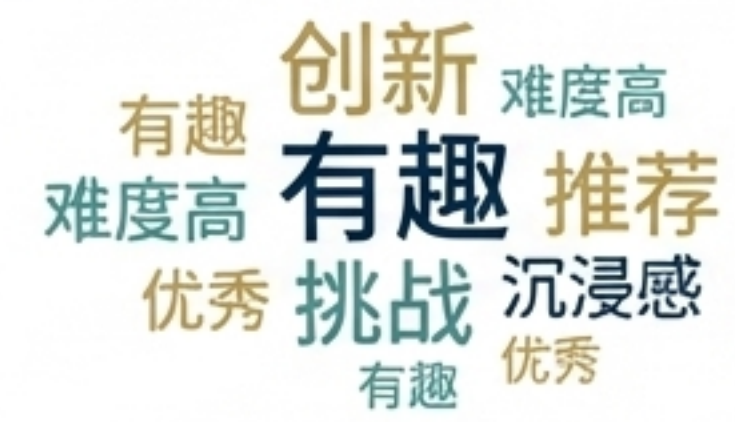
156

发行商视角

社区热度趋势 (H_dynamic)



玩家评价词云



技术驱动创新，数据连接未来。